





Introduction

- Formula 1 cars generate large quantities of telemetry
 - This project develops a Python-based F1 telemetry analysis and visualization system.
 - A simplified aerodynamic model estimates downforce and drag from vehicle speed.
- 



Problem Statement

- The problem addressed by this project is to create a simple software system that converts raw F1 telemetry into useful visual information
 - Objective: Develop a reproducible Python application that loads an F1 session, extracts a driver's fastest lap, processes telemetry,
- 



Functional Requirements

- Telemetry loading - Load a selected Formula 1 session using FastF1.
- Session selection - Select season, Grand Prix, session type and driver.
- Telemetry processing Extract and clean speed, throttle, brake, distance, X/Y and related channels.
- Aerodynamic calculations Estimate downforce and drag using simplified aerodynamic equations.
- Visualization Generate - telemetry graphs and a dashboard.
- Animation - Animate vehicle position and changing telemetry values.
- Error handling - Handle missing data, unavailable sessions and invalid telemetry selections.
- Output - Display calculated values and visual results for interpretation.

Non-functional Requirements

- Performance: - The application should process a normal telemetry lap within a standard student computer.
- FastF1 caching should reduce repeated downloads.

Reliability: The program should handle missing values or unavailable telemetry channels without producing misleading output.

- Maintainability: Data loading, calculations, visualization and animation should be organized into logical sections/functions.
- Portability: The project should run in a standard Python environment with the required packages installed

System Architecture

- The system follows a data-processing pipeline. FastF1, Pandas and Matplotlib generates graphs and animation.



Design Diagrams

- Use Case Diagram
Select F1 session → Load session data
View results ← Calculate aero + generate graphs
Start animation ← Update dashboard frames
- Workflow Diagram
Start → FastF1 → Load session → Select driver
→ Pick fastest lap → Calculate derived quantities → Plot dashboard → Animate → Display results → End.

- Sequence Diagram
User Python Program FastF1 Matplotlib
Select session → Request session → Return session
Run analysis → Load telemetry → Return telemetry
Process/filter data
Calculate aero
View dashboard → Send processed arrays → Render plots
Start animation → Update frame → Render frame

- Component / Class Diagram

Session Loader Creates FastF1 session and loads timing/telemetry data.

Lap Selector Filters the requested driver and selects the fastest lap.

Telemetry Processor Cleans data and converts selected columns into arrays.

Aero Model Calculates downforce and drag from speed and assumed parameters.

Dashboard Creates text, plots and telemetry indicators.

Animation Controller Updates plotted vehicle/telemetry state for each frame

- ER Diagram
Not applicable



Design Decisions & Rationale

- FastF1 was selected because it provides a convenient Python interface for accessing F1 timing and telemetry data.
- Pandas was selected because telemetry is naturally represented as tabular data.
- NumPy supports efficient numerical arrays and calculations.
- Matplotlib supports both scientific plots and animation.
- .

Implementation Details

- The core implementation uses Python with FastF1, Pandas, NumPy and Matplotlib.
- The current project work includes a Monaco Grand Prix 2024 race session, selection of driver LEC (Charles Leclerc), fastest-lap telemetry extraction, etc.
- Session loading and fastest-lap extraction
`session = fastf1.get_session(2024, "Monaco Grand Prix", "R")`
- Telemetry cleaning
- Aerodynamic calculations
Downforce: $F_{\text{down}} = 0.5 \times \rho \times C_L \times A \times v^2$
Drag: $F_{\text{drag}} = 0.5 \times \rho \times C_D \times A \times v^2$
- Animation
Matplotlib's FuncAnimation

Screenshots

Gear statistics:

```
count    570.000000
mean      3.936842
std       1.807875
min       1.000000
25%       2.000000
50%       4.000000
75%       5.000000
max       7.000000
```

Name: nGear, dtype: float64

=====

FORWARD SIMULATION

=====

```
Simulated lap time: 38.600 s
Simulated top speed: 336.5 km/h
Simulated distance: 3284 m
```

=====

=====

F1 PHYSICS MODEL

=====

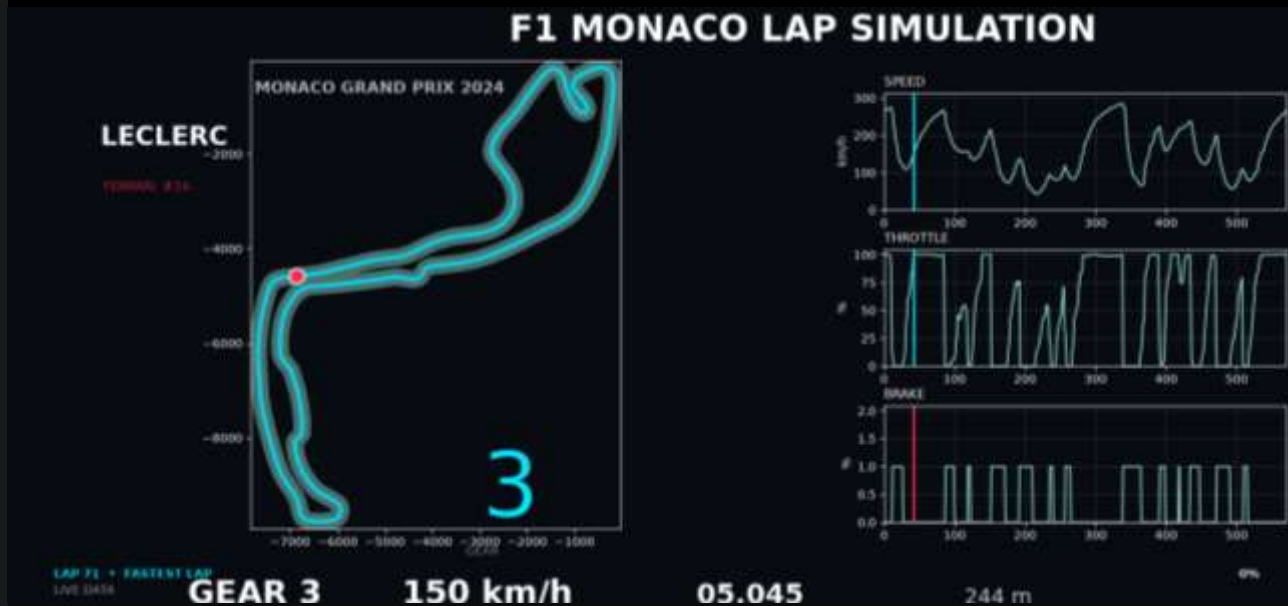
```
Maximum downforce: 17274 N
Maximum drag:      5758 N
Maximum tyre force: 40196 N
Peak acceleration: 22.30 m/s2
Peak acceleration: 2.27 G
```

=====

LAP ANALYSIS

=====

```
Track length: 3283 m
Average speed: 159.0 km/h
Top speed: 285.0 km/h
Braking time: 0.00 s
Full throttle time: 25.32 s
Lap time: 75.162 s
PS C:\Program Files\Microsoft VS Code> |
```



Testing Approach

- Import FastF1 Library imports without error.
- Select driver Driver laps are returned.
Pick fastest lap A valid fastest lap is selected
- Aero calculation Downforce and drag return numerical values for valid speed.
- Plot generation Graphs render with labels and titles.
- Animation Dashboard updates without runtime errors.
- Invalid session Program reports an error

Challenges Faced

- FastF1 data loading -Sessions can take time to download and depend on data availability and caching.
- Running from the VS Code installation directory caused a file-not-found error
- Missing telemetry values Rows without valid X,Y speed values are filtered before visualization
- Animation debugging FuncAnimation requires correct frame updates, object references and syntax



Learnings & Key Takeaways

- The project provides a foundation for future development. A more advanced version could include a complete lap-time simulator , multiple-driver comparison, circuit-specific aerodynamic settings and optimization of vehicle parameters
- 



Future Enhancements

- • Physics-based acceleration and braking model
 - Driver-to-driver telemetry comparison
 - Sector and corner-by-corner performance analysis
 - Track comparison such as Monaco vs. Monza
 - Machine-learning-based lap-time or setup analysis



References

- FastF1 Python package documentation
- Pandas documentation — data structures and data analysis tools.
- NumPy documentation — numerical computing in Python.
- Matplotlib documentation — plotting and animation.